WORKING SESSION · VULNERABILITY REMEDIATION · OCTOBER 2026

Clear the repeatable work. Keep engineering judgment with the team.

A practical look at today’s remediation flow, where the effort goes, and how a small, deterministic accelerator could take on the predictable parts — inside the existing process, with every release still approved by people.

- Process first
- Verified, not guessed
- Start small
- Human-governed release

The live demo uses purpose-built demo repositories — no customer systems, credentials or data.

TODAY’S 45 MINUTES

A conversation first, a demo in the middle, discussion at the end

The session begins with the current process, followed by a practical demonstration and discussion of potential areas for improvement.

01
Current process
A working view of the end-to-end flow
~5 min

02
Where the effort goes
Dependencies, repetition, low-value effort, inconsistency
~6 min

03
Approach and safeguards
Simple, safe, start small; where testing happens
~7 min

04
Ways to use it
End to end, or one piece at a time
~3 min

05
Live demo
Scenarios, run output, Command Center, a real MR
~14 min

06
Wrap-up and discussion
Boundaries, success measures and potential value
~8 min



What would help most: Identify areas where the current process could be improved, and discuss which of the proposed ideas could make a meaningful difference to the team.

A large estate, a steady stream of findings, and knowledge spread across many people

The vulnerability team fixes applications it does not own day to day. Many fixes depend on context held by different leads and SMEs.

140+
application assets in scope for remediation

~50K
findings in the regular scanner (Qualys) report, OS and application together

2
main stack families: Java (Gradle/Maven) and Node.js (npm/Yarn)

Manual
most steps today: filtering, assignment, fixing, building, testing, MRs and release

Different per asset
Base-branch naming, build setup, how to run the service, and how to test it all vary by application and by SME.

Partial context
Vulnerability developers work only on fixes, so they rely on leads and SMEs for clarifications — across onshore and offshore teams.

Mixed findings
Red Hat OS-layer and application findings arrive together; only application dependencies are fixed in code.

> The question for discussion: where can repeatable preparation and coordination effort be reduced **without weakening engineering quality or release control?**

Today’s flow: sixteen steps across three phases

PHASE 1

Triage and assign

1. Scanner report generated regularly
   OS and application findings together
2. Apply filters by hand
   Grouping (OS vs application), cluster, environment, namespace, severity routed to teams
3. Assign assets to developers

PHASE 2 · REPEATS FOR EVERY ASSET

Remediate

4. Identify the base branch
   Naming pattern differs by SME and team
5. Clone or pull; create a feature branch
6. Build and compile the untouched code locally
7. Fix: direct upgrade, transitive upgrade, or code change
8. Bump `build.properties` where used
9. Rebuild with Gradle, Maven, npm or Yarn
10. Run and test locally
    Setup differs per app; may need databases, external APIs, Bruno collections, tokens
11. Raise a merge request

Throughout: ask the lead or SME — which branch? how does it build? how do I run and test it?

PHASE 3

Release

12. Review and merge to the base branch
13. Deploy to Staging with Jenkins
14. Update the image tag on the other channel deployments
    Same base code, several routes
15. Test in the environment with testers
16. Code freeze, production release and security rescan

Orange numbers: steps that repeat almost identically for every simple fix. Phase 3 is an intentional human-controlled boundary.

Much of the effort comes before the fix itself

Observed activity	Type	Why it adds effort	Potential improvement
SME input and cross-team handoffs	`Dependency`	Branch, build, run and test knowledge sits with individuals; handoffs across time zones cause delays	Maintain SME-verified repository and branch settings in a registry; share each asset's execution outcome and blockers
Setup and baseline build for every asset	`Time-consuming`	Clone, branch and build repeat for each assigned asset before any fix starts	Run the first build of all assigned assets unattended and record each outcome
Complex local run-and-test setup for minor bumps	`High setup effort`	Needs databases, APIs, tokens and Bruno setups that differ per app — for a patch-level change	Build and unit tests before the MR; thorough testing in Staging with testers (to be agreed)
Different patterns	`Inconsistency`	Branch naming, environment setup and test approach vary; results depend on who did the work	Same rules on every run; one tracker with a state and a reason for every finding
Build issues identified late	`Rework`	An already-broken build or a breaking upgrade shows up only after the setup effort is spent	Build the untouched code first; upgrade in small batches and roll back only what fails
Status spread across chats and spreadsheets	`Visibility`	Managers and offshore teams ask "where is this?" and "why did it stop?"	Dashboard and trace generated from the same tracker the tool writes

> Common thread: **much of the effort is context and setup, repeated per asset** — rather than the version change itself.

What changes, and what stays exactly as it is

Step	Today	Proposed	Responsibility
Filters and assignment	Re-applied by hand each cycle	Configured once in templates; developer-specific files generated	Tool
Repository and base branch	Asked of the SME; patterns vary	Registry holds the GitLab URL and base branch, with per-asset overrides; verified before cloning	Tool
Clone, branch, baseline build	By hand, per asset	Automated; a failure is recorded and the run moves on	Tool
Simple dependency upgrades	Edited by hand	Applied in small batches; configured version bump where applicable	Tool
Rebuild and verify	By hand	Native Gradle / Maven / npm / Yarn build and configured tests; pass committed, fail rolled back	Tool
Code changes, breaking upgrades	Developer, after setup	Developer — handed over earlier, with the build output attached	People
Merge request	Written by hand	Passing changes become MR-ready; MR created or confirmed by configured mode, one per asset	Tool
Review, merge, Jenkins, channel tags, testing, code freeze	Manual	**Unchanged** — human in the loop	People
Status and coordination	Chats, emails, spreadsheets	Tracker and Command Center trace, per finding	Shared

The line sits at the merge request. Everything after it remains the team's decision.

Automate what can be verified. Escalate what requires engineering judgment. Report everything.

Small, deterministic and safety-first — deliberately not a universal vulnerability-fixing engine.

Design choices

- ✓ **Deterministic rules, no AI guessing.** Explicit rules give consistent decisions from the same inputs and repository evidence.
- ✓ **Configured in Excel** the team already uses — readable by developers and managers.
- ✓ **Uses existing tools:** Git, GitLab, Gradle, npm/Yarn, run from a workstation.
- ✓ **Stops rather than guesses** when evidence is missing; never falls back to another branch.

Deliberately not building

- ✗ An end-to-end platform with its own infrastructure and learning curve
- ✗ Automatic merge, deployment or closure of vulnerabilities
- ✗ Automated code refactoring or framework migrations
- ✗ A tool that must be adopted all at once to be useful

> Start with one piece, measure whether it helps, and keep it only if it does.

One pass per asset: prove each step, or stop and record why

01
Read and filter
Master report plus configuration templates in; filters applied, developer-specific files out.

→

02
Verify base branch
The configured release branch, or the asset's override, must exist in GitLab.

◆ Missing → blocked, no fallback

→

03
Baseline build
Clone and build the untouched code with its own build and tests.

◆ Already broken → nothing changed

→

04
Confirm versions
The scanner's version must match the build file or lockfile before any edit.

◆ Mismatch → handed back as drift

05
Branch and bump
One feature branch per asset; configured version bump where applicable.

→

06
Upgrade in small batches
Two libraries at a time; rebuild and retest after each batch.

◆ Fails → that batch rolled back

→

07
One MR per asset
Passing changes are pushed and become MR-ready; MR creation depends on mode. Reruns reconcile existing work.

→

08
Record everything
State, reason, branch, commit, MR status and relevant diagnostics for every finding — then the next asset.

Any deviation is recorded and the run continues with the next finding or asset. **It is designed to run unattended.**

Today’s filters and assignments become templates — set once, applied every cycle

Scope filters

- **Grouping:** Red Hat OS vs. application
- **Cluster:** the physical cluster an asset runs on
- **Environment:** dev, test, stage, prod
- **Namespace:** application groups in scope
- **Asset:** include or exclude specific assets

Routing and ownership

- **Severity to team:** e.g., medium and low to one team, high and complex to another
- **Asset to developer**
- Output: one file per developer

Asset registry

- Exact GitLab repository URL
- Release base-branch pattern
- Per-asset branch override when the team’s naming differs
- Base asset and its channel deployments

Run settings

- Batch size for upgrades
- `build.properties` bump where present
- Merge-request behaviour
- Resume pending work, or rebuild from a fresh report

Onboarding a new asset is **a row in a sheet**, not a code change. The registry also becomes the shared record of “which branch, which repo” that today lives with SMEs.

No row is dropped: excluded and OS-layer rows **stay in the tracker with the reason**, so every row in the report is accounted for.

Place detailed testing where it is most effective — not remove it

A discussion proposal, not an agreed change. Testing depth should reflect the risk of each change and existing requirements.

ALWAYS · BEFORE THE MR

Automated checks

- Untouched code builds first
- Native build passes; configured tests run where present
- Updated dependency version verified using repository evidence
- Evidence and build output attached

CODE CHANGES · BREAKING UPGRADES

Developer tests locally

- Same as today, where it matters most
- Framework or major-version changes
- Anything touching application code
- Apps with no viable Staging test path

MINOR · PATCH BUMPS

Thorough test in Staging

- Deploy via Jenkins as today
- Testers run API, UI and regression checks
- One good round in a real environment, instead of a partial local setup per app

To agree together: **which changes qualify** for Staging-first testing, and **what evidence an MR must carry** for a reviewer to trust it.

Use it end to end, or one piece at a time

Different outputs can support different steps. Start where repeated effort is greatest.

A

Triage and assignment

Apply the filter and routing templates to each new report; generate developer-specific files.

Saves: repeated manual filtering every cycle

B

First-run build of all assets

Clone and baseline-build eligible assigned assets as part of a run; record passes and failures.

Saves: local setup just to learn the starting point

C

Simple fixes to MR

Eligible direct upgrades in batches, configured build/tests, and a review-ready branch or MR.

Saves: mechanical edits and MR preparation

D

Trace and dashboard

Per-asset trace of what happened; quick status by state, owner and severity.

Helps: onshore/offshore handoffs and status calls

E

Process standardization

Document filters, branch patterns and test policy to identify where consistency can improve.

Helps: streamline, even outside the tool

One possible first step is **B + D**: see the build state of each assigned asset early, and share it without a separate status meeting.

One run, 25 seeded findings, a range of realistic scenarios

Six purpose-built GitLab repositories with findings designed to exercise the happy path *and* exception cases.

MOVES FORWARD	STOPS SAFELY	ROUTED BY RULE	REPORTED
Direct upgrades — Java and Node Declared version updated; batch passes; MR prepared or created by mode	**Base branch missing** Blocked; never falls back to main	**Channel / replica deployments** Linked to the base asset; fixed once	**Every finding has a state** And a plain-language reason
Node transitive fix Package pulled in by another library; pinned by a Yarn resolution	**Baseline already broken** Halts before changing anything	**Exact duplicates** Linked, not reprocessed	**Build output preserved** For any halt or rollback
Per-asset branch override Team uses its own base-branch name	**Version cannot be proved** Finding not edited; evidence recorded	**Excluded by filter** Cluster, namespace, asset, severity	**Safe to rerun** Same branch, existing MR reused
Mixed outcomes in one asset Passing changes are prepared for one MR; the rest requires follow-up	**Upgrade breaks the build** Only that batch rolled back	**Red Hat OS packages** Routed to the base-image path	**Real merge requests** Open in GitLab, none merged by the tool

Demo repositories and seeded data only; no customer systems or data are used.

What the demo will cover

1

Inputs

- Master scanner report
- Configuration tabs: filters, exclusions, registry

2

Run output

- Tracker: state, reason, branch, commit, MR per finding
- Developer-specific view

3

Command Center

- Overview by state, owner, severity
- Per-asset trace: happy path, then the safe stops
- Search and export

4

An example GitLab MR

- Small, readable diff
- Single version bump
- Correct target branch

Questions to keep in mind while watching

Would this trace help communication in status calls and onshore/offshore handoffs?

Which of the “stops safely” cases happen most often in the estate?

What would a reviewer need to see in the MR to approve it quickly?

Visibility is valuable even when nothing can be fixed automatically

Vulnerability Command Center — Illustrative layout · no real values

Current status
Findings by state, owner and severity

Blockers
Each blocker, with its reason

Next action
Review, retry or escalate

State trace — one asset

Branch verified → Baseline built → Versions confirmed → Batch 1 passed → Batch 2 rolled back → MR raised

Diagnostic
What happened · why it stopped · who acts next · which branch, commit or MR holds the evidence

For the IT manager
Distribution of work, outcomes, ownership and blockers — without reading build logs.

For developers and SMEs
The failing step, its build output, and the branch or MR to pick up from.

For onshore / offshore handoffs
One shared trace instead of reconstructing status across shifts and chats.

The dashboard renders the Excel tracker the tool writes: a snapshot view, not a live GitLab sync or a closure engine.

Clear about what it does — and what stays with people

The tool does

- ✓ Filter, route and assign from templates
- ✓ Verify branch and baseline build
- ✓ Direct upgrades; Node transitive overrides
- ✓ Build, test, commit or roll back
- ✓ Prepare one MR per asset; create where configured
- ✓ Record every outcome

Stays with people

- ✗ Review, approve and merge
- ✗ Jenkins deployment and channel image tags
- ✗ Functional and regression testing
- ✗ Code freeze and production release
- ✗ Code changes and framework migrations
- ✗ Confirm closure by rescan

Handed back for now

- Versions managed centrally (BOMs, version catalogs)
- Multi-module builds
- Java transitive constraints
- Anything the tool cannot prove from the repository

Flagged in the tracker for a developer — never guessed.

An MR is not a fix. A finding is closed only after it is merged, deployed, tested and confirmed by a rescan — all under the existing process.

What success looks like — and how to measure it

AUTOMATION

Prepare straightforward fixes

Prepare verified, reviewable MRs for straightforward dependency upgrades.

DEVELOPER SUPPORT

Surface complex cases

Blockers, failed builds, version drift and manual work — with actionable diagnostics.

MANAGER VISIBILITY

Track every finding

Excel tracker and Command Center show outcome, owner and reason.

SAFETY

Never silently change production

MR approval, merging and deployment stay human-controlled.

Measure any trial against an agreed baseline

Time to a review-ready MR

From valid finding to an MR, or a clearly documented blocker

Problems caught before review

Broken baselines and breaking upgrades found early

Developer touchpoints

Hours on triage, lookups, setup and routine edits

Clarity of ownership

Fewer repeated requests for the same context

Regressions after merge

No increase relative to the agreed baseline

Verified closure

Findings confirmed fixed by the post-release rescan

> A count of MRs is never reported as vulnerabilities closed. Start from an agreed manual baseline and a small sample.

Where would one small improvement help the team most?

1. Which part of today’s flow takes the most time?
   Triage, setup and builds, testing, coordination?
2. Which assets would make good first candidates?
   Low risk, builds cleanly, an owner willing to try
3. What must an MR carry for a reviewer to trust it?
4. When would Staging-first testing be acceptable for a minor bump?
5. Would a shared trace improve onshore/offshore coordination?

OPTIONAL · IF THERE IS INTEREST

A small, measurable trial

- **A handful of low-risk assets** chosen with the leads
- **One or two pieces** — for example the first-run build and the trace
- **Agreed up front:** success measures relative to the current process
- **A decision afterwards** to continue, change or stop

A follow-up session can explore selected areas in greater detail.

POSSIBLE STARTING POINTS

Intake and ownership
Make recurring triage and assignment consistent

First-run builds
See each assigned asset’s build state and blockers early

Handover clarity
A shared trace that makes the next action explicit

Selected simple fixes
Review-ready changes for a narrow set of upgrades